



UNIVERSIDAD DE LOS LAGOS

9

# Programación Orientada a Objetos

Docente: Patricio Araneda García

Correo: [patricio.aranedagarcia@ulagos.cl](mailto:patricio.aranedagarcia@ulagos.cl)



**PATRICIO ARANEDA GARCÍA**  
Magíster en Informática Médica  
Universidad de Chile

DATACLINICA Spa



patricioaraneda.cl



dataclinica.cl



/in/patricioaraneda



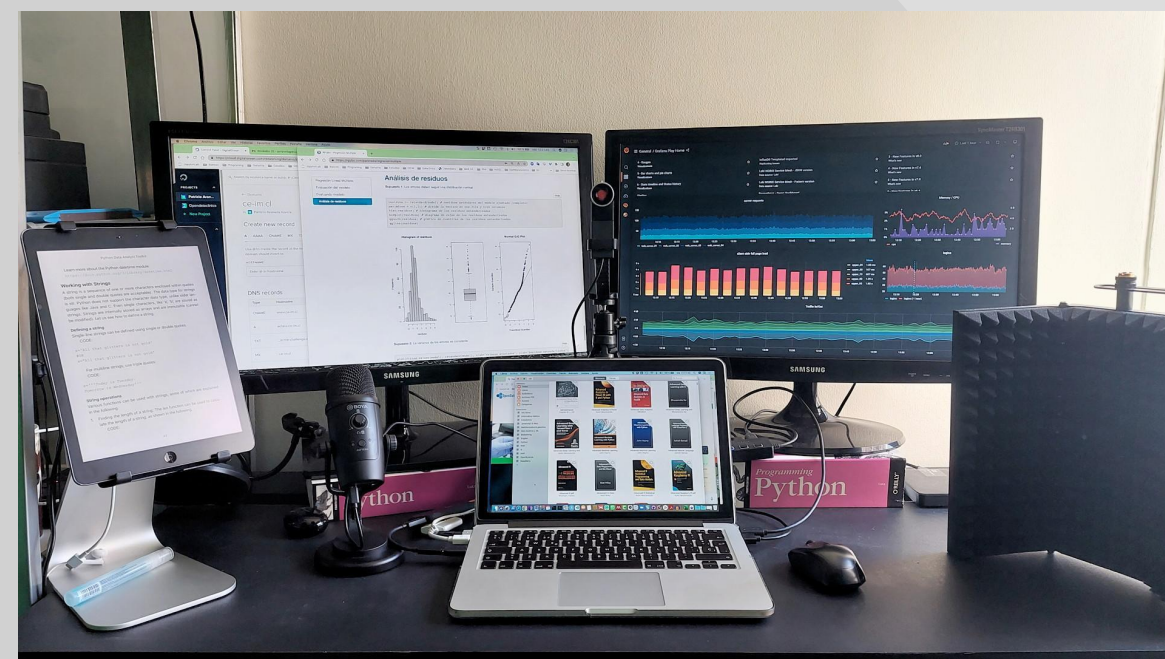
@paranedagarcia

**Profesor de Ciencias Naturales y Biología**  
Pontificia Universidad Católica de Chile

**Ingeniero en Computación e Informática**  
Instituto Profesional Virginio Gómez

**Magíster en Informática Médica**  
Universidad de Chile

- Salud Pública y Redes Asistenciales
- Inteligencia Artificial
- Modelamiento Productivo y Machine Learning



<https://github.com/paranedagarcia>



<https://rpubs.com/paraneda>

## DOCENCIA UNIVERSITARIA

- Análisis de Datos
- Bases de Datos
- Fundamentos de Sistemas Inteligentes
- Sistemas y Tecnologías de Información
- Ciencia de Datos y Machine Learning
- Inteligencia Artificial
- Bioestadística



UNIVERSIDAD DE LOS LAGOS

# CONTENIDOS DEL CURSO

## Manual de Ingeniería Informática

Desarrolla sobre las Tecnologías de la Información

Manual



### Fundamentos

Conozca los conceptos básicos de la Ingeniería de la Información y aplique para la resolución de problemas en cualquier ámbito.



### Programación

Aprenda las bases de la programación, algoritmos y estructuras de datos, incluyendo la sintaxis, estructuras de control y conceptos fundamentales.



### Gobernanza

Aprenda las mejores prácticas para la gobernanza de los datos e información en el ámbito organizacional (en construcción...).

#### Otros cursos de la serie

- Ingeniería Informática
- Programación en Python
- Bioestadística
- Ciencia de Datos
- Análisis con SQL

#### Conecta

- Website
- LinkedIn
- Whatsapp

#### Más

- GitHub
- ORCID
- Correo

#### Licencia

CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!

## Programando con Python

Desde Ciencia de Datos hasta Machine Learning

Curso Python



### Programación

Este curso fue diseñado desde cero para ser fácilmente asquible y comprensible para todos. No requiere conocimientos previos de estadística o programación.



### Data Science

Se enfoca en la aplicación de la estadística en la investigación médica, y cubre los conceptos y técnicas estadísticas más relevantes para el análisis de datos en este campo.



### Desarrollo Continuo

Esta en constante desarrollo, y se irán agregando nuevos contenidos y recursos a medida que se avance en el curso. Se anima a los estudiantes a participar activamente en el curso, compartiendo sus experiencias y sugerencias.

#### Otros cursos de la serie

- Ingeniería Informática
- Programación en Python
- Bioestadística
- Ciencia de Datos
- Análisis con SQL

#### Conecta

- Website
- LinkedIn
- Whatsapp

#### Más

- GitHub
- ORCID
- Correo

#### Licencia

CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!

<https://patricioaraneda.cl/informatica>

- Programación

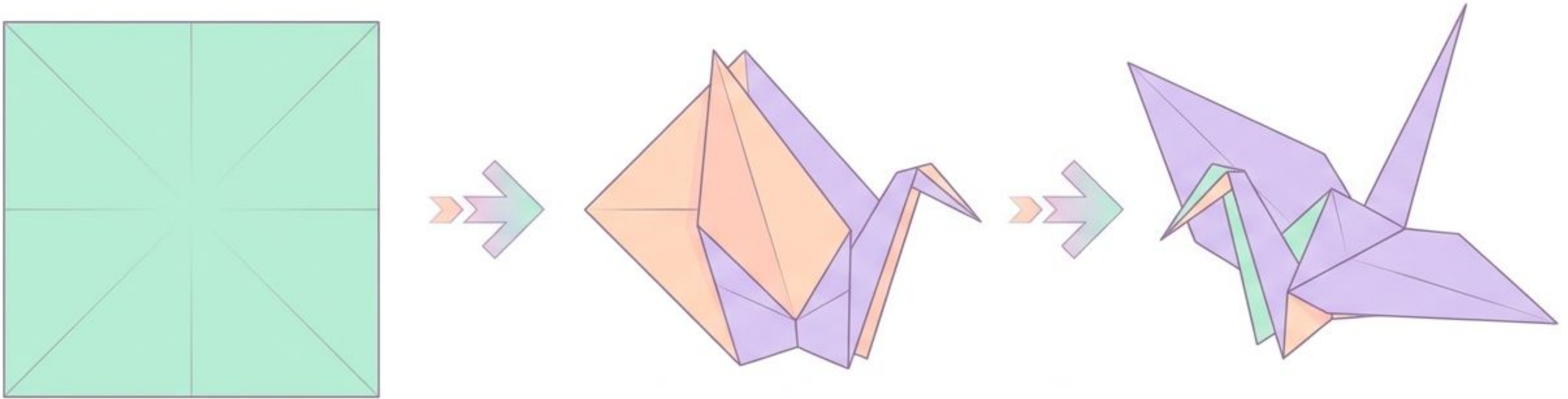
<https://patricioaraneda.cl/python/docs/poo/>

The background is a complex digital illustration. It features several server racks with glowing blue lights. In the center, there is a network of glowing green spheres connected by thin, translucent lines. A bright, glowing orange and yellow sphere is at the center of this network. The background is filled with abstract digital patterns, including lines of code in blue and orange, and various mechanical gears and circuit components. The overall color scheme is dominated by blue, green, and orange.

# Polimorfismo

# Polimorfismo en POO

El Arte de las Múltiples Formas en Python



# ¿QUÉ SIGNIFICA “POLIMORFISMO”?



Poly (Muchos)



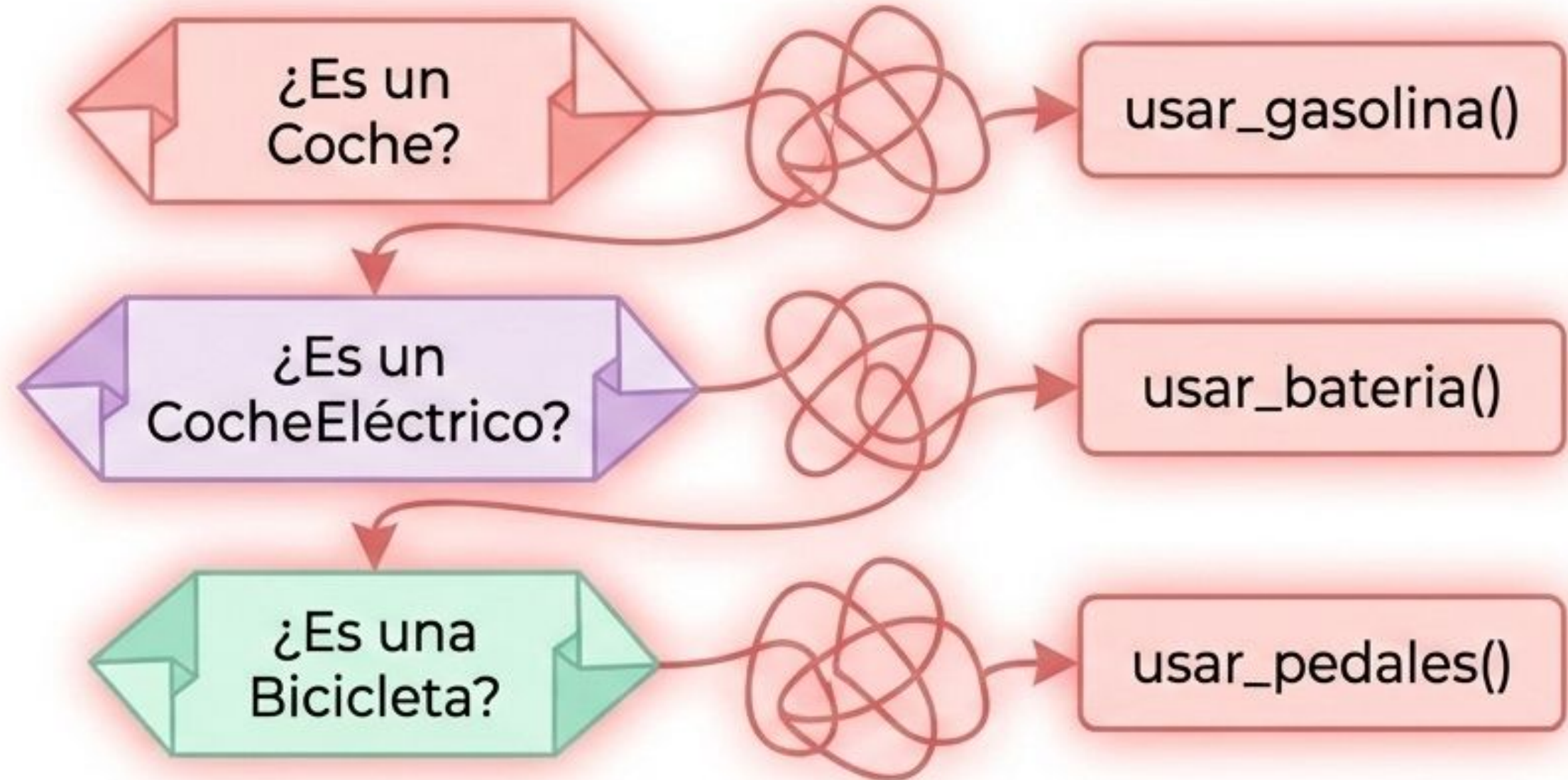
Morphos (Formas)



Un mismo mensaje,  
múltiples respuestas

La capacidad de tratar objetos de diferentes clases de manera uniforme.

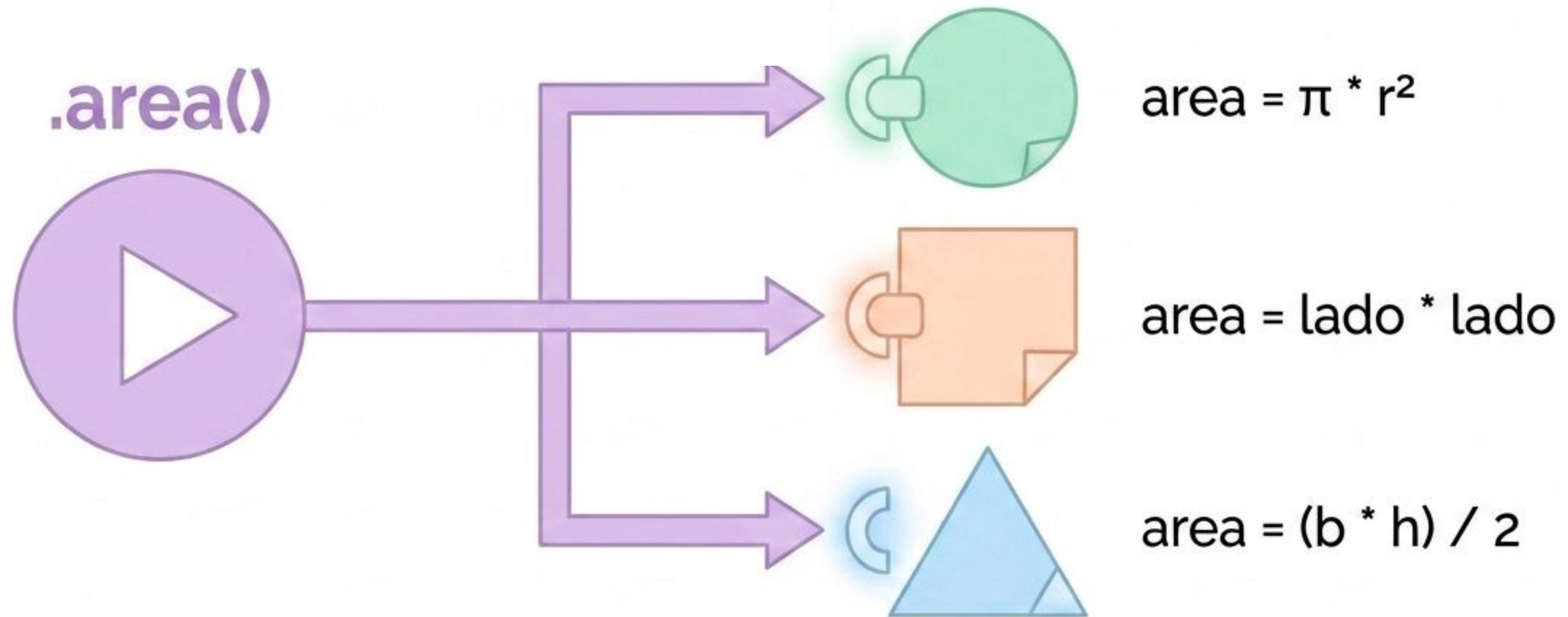
# EL PROBLEMA: CÓDIGO RÍGIDO Y FRÁGIL



Saturar el código preguntando constantemente "¿qué tipo de objeto eres?" genera sistemas ineficientes y difíciles de mantener.

# LA SOLUCIÓN: INTERFAZ ÚNICA

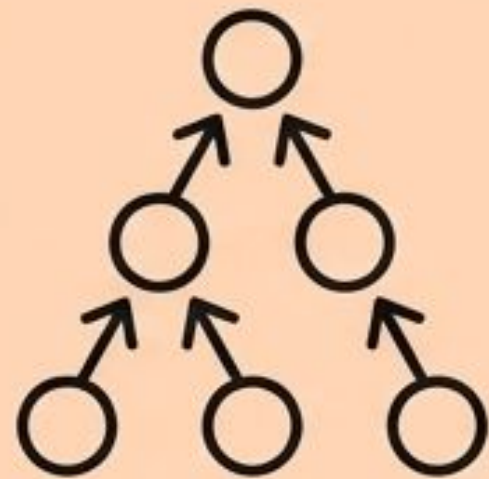
El adaptador universal



Simplemente llamamos a `.area()` y confiamos en que el objeto sabe cómo responder.

# LAS 3 CARAS DEL POLIMORFISMO EN PYTHON

Tres mecánicas diferentes, un mismo objetivo: crear sistemas modulares



**1. Herencia y  
Sobrescritura**



**2. Duck Typing  
(Tipado de Pato)**



**3. Sobrecarga de  
Operadores**

# SOBREESCRITURA DE MÉTODOS (OVERRIDING)



Las subclases redefinen (sobrescriben) el método de su clase padre. Si llamas al método en la subclase, la nueva versión toma el control.

# EL CODIGO: SOBREENSCRITURA EN ACCIÓN

```
for figura in [Circulo(5), Cuadrado(4)]:  
    print(figura.area())
```



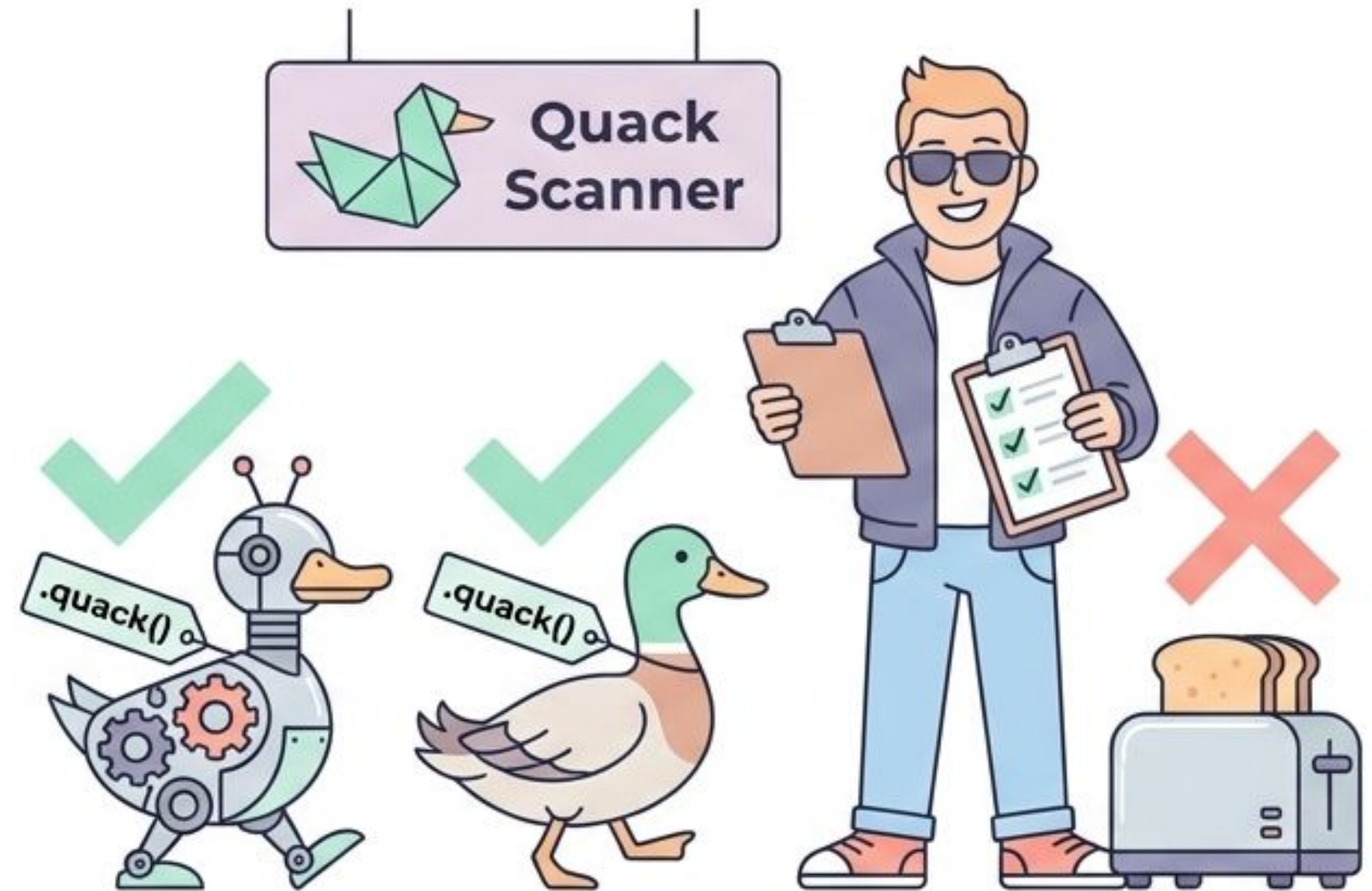
El bucle no necesita saber qué figura exacta está procesando. El polimorfismo se encarga de ejecutar la fórmula correcta automáticamente.

# DUCK TYPING (TIPADO DE PATO)

## Java/C++



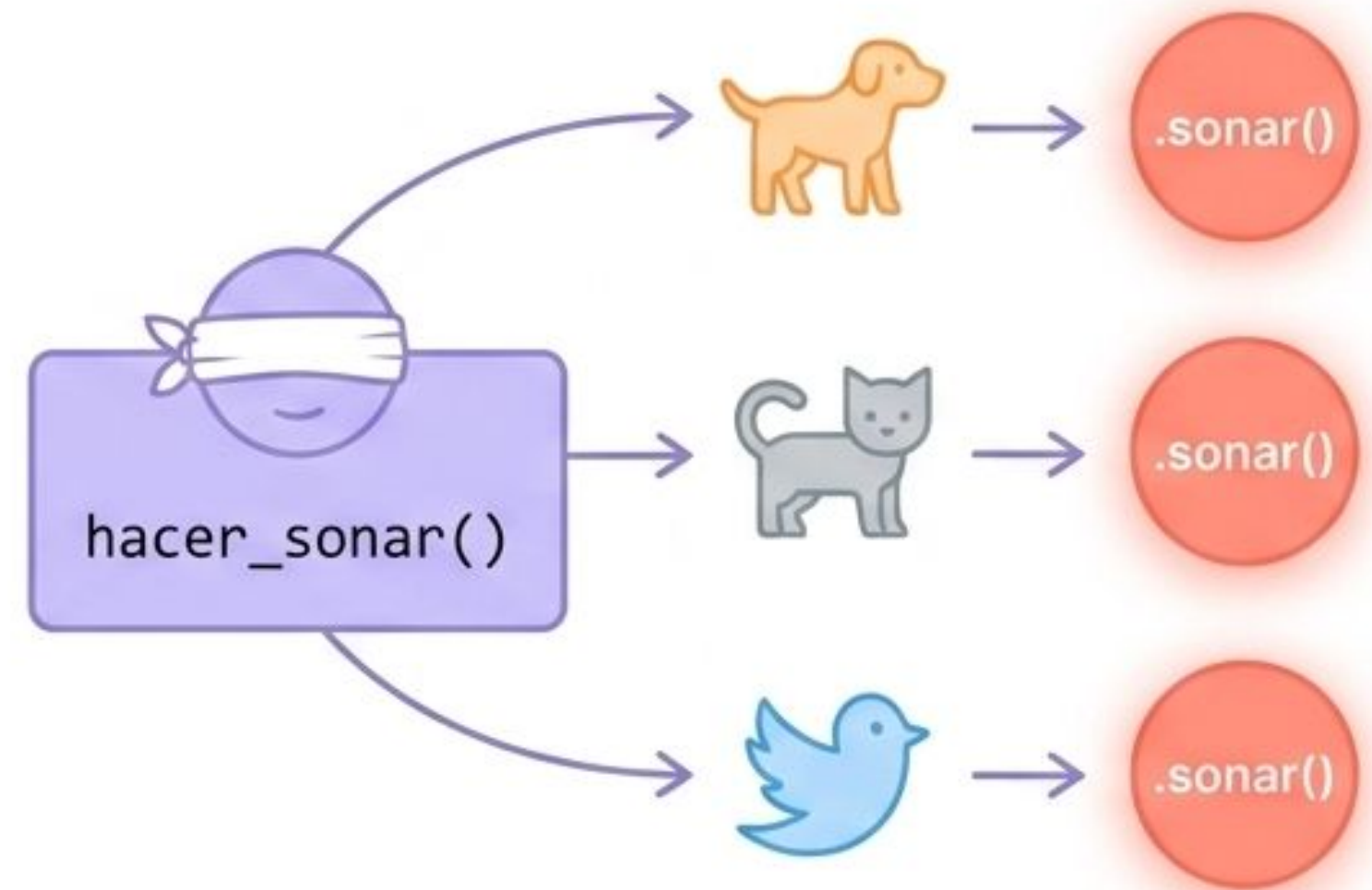
## Python



Si camina como un pato y grazna como un pato... Python lo trata como un pato.  
No importa el tipo de clase, solo importa el comportamiento (métodos).

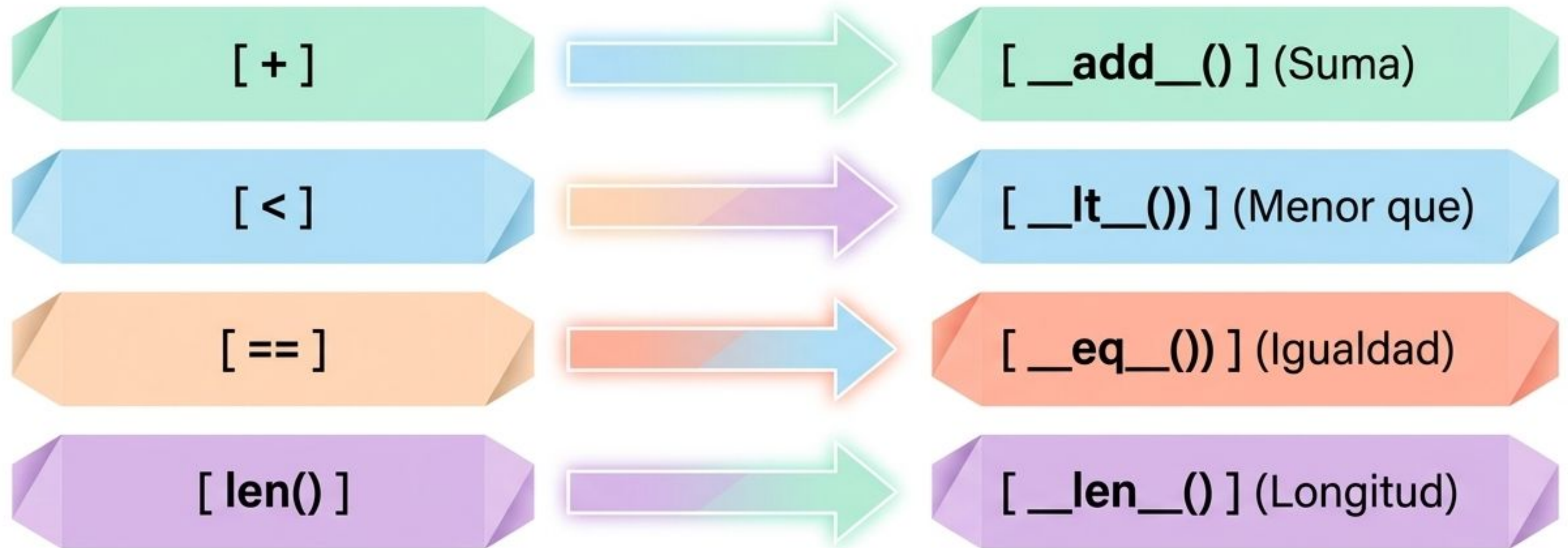
# El Código: Confianza Ciega

```
def hacer_sonar(animal):  
    animal.sonar()
```



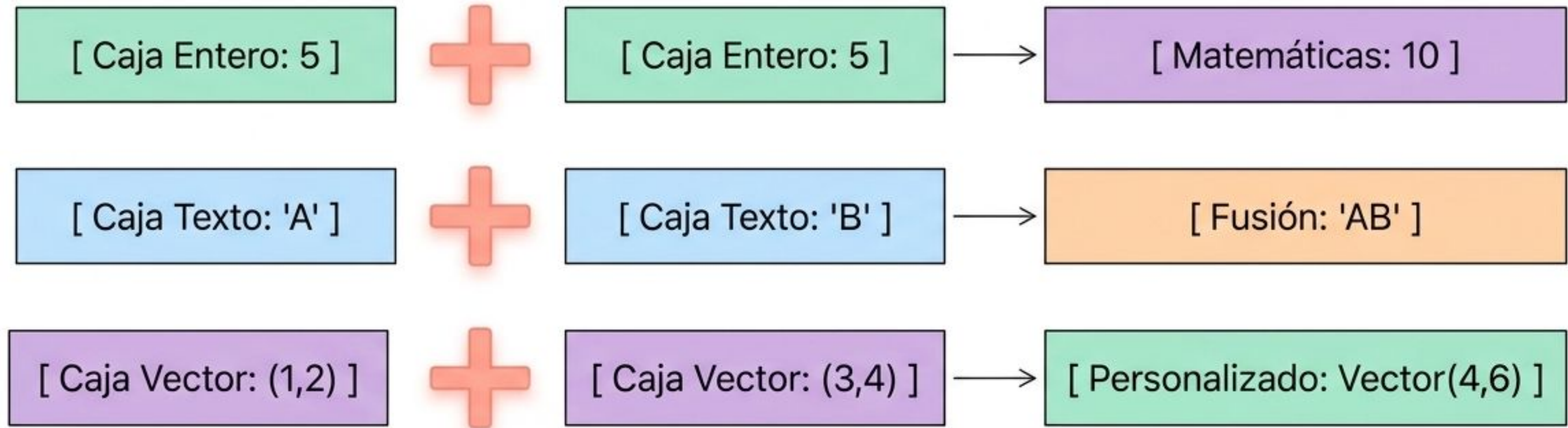
En Python, no verificamos el tipo exacto del argumento. Programamos asumiendo que el objeto soportará la interfaz requerida.

# SOBRECARGA DE OPERADORES (MAGIC METHODS)



Los operadores estándar (+, -, ==) son polimórficos. Su comportamiento cambia dependiendo de los 'Métodos Dunder' (Double Underscore) definidos en la clase.

# Un Operador, Múltiples Realidades

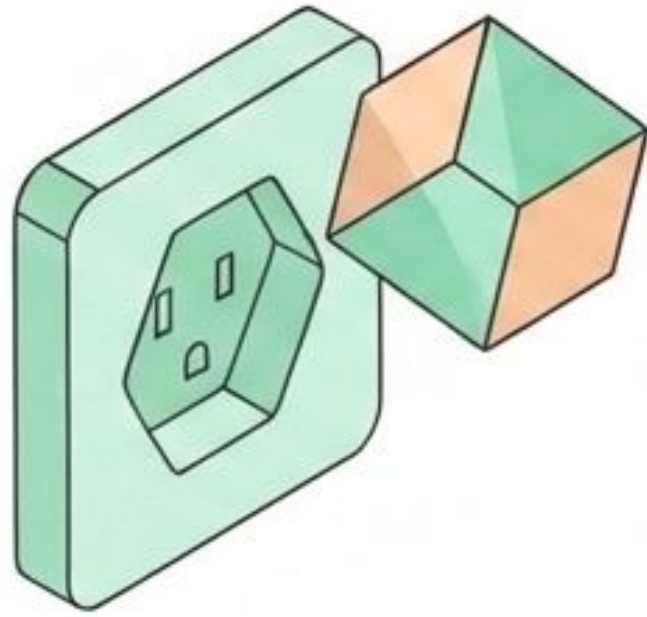


El signo '+' no suma. Simplemente envía un mensaje `__add__`.  
Es el objeto quien decide qué significa ese mensaje.

# Matriz Diagnóstica: Elige tu Polimorfismo

| Mecanismo     | Paradigma              | Gatillo (Trigger)        | Caso de Uso Principal                               |
|---------------|------------------------|--------------------------|-----------------------------------------------------|
| Sobrescritura | Formal / Estricto      | Llamada a método         | Jerarquías de herencia (Clases Padre/Hijo).         |
| Duck Typing   | Informal / Flexible    | Llamada a método         | Objetos no relacionados que comparten una interfaz. |
| Sobrecarga    | Sintáctico / Intuitivo | Uso de Operadores (+, *) | Matemáticas personalizadas y clases de utilidad.    |

# ¿Por qué el Polimorfismo es tan Poderoso?



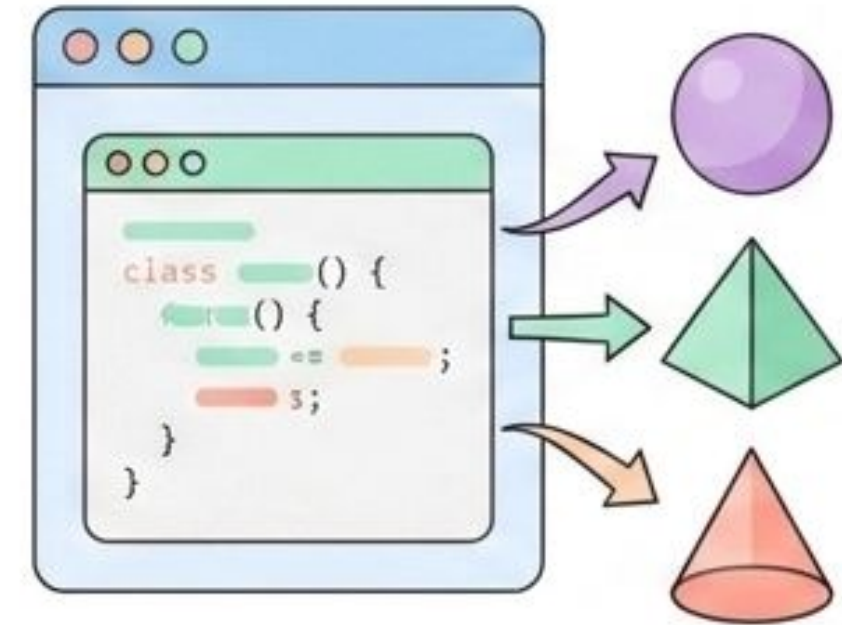
## Plug & Play (Extensibilidad)

Nuevas clases pueden agregarse sin alterar el código central.



## Abstracción

Ocultas los detalles complejos de implementación.



## Reutilización

Escribes menos código, pero logras más impacto.